



Strongly Connected Components and Topological Sort.

1 Topological Sort

Definition 1. (connectivity)

1. Let $G = (V, E)$ be a non-directed graph. A **connected component** of G is a subset $U \subseteq V$ of maximal size in which there exists a path between every two vertices.
2. A non-directed graph G is said to be a **connected** graph if it only has one connected component.
3. Let $G = (V, E)$ be a directed graph. A **strongly connected component** of G is a subset $U \subseteq V$ of maximal size in which for any pair of vertices $u, v \in U$ there exist both directed path from u to v and a directed path from v to u .

1.1 Depth First Search (DFS)

As its name implies, depth-first search searches "deeper" in the graph whenever possible. Depth-first search explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it. Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered. This process continues until all vertices that are reachable from the original source vertex have been discovered. If any undiscovered vertices remain, then depth-first search selects one of them as a new source, repeating the search from that source. The algorithm repeats this entire process until it has discovered every vertex.

```

1 DFS(  $G$  ):
2   for  $v \in V$  do
3      $v.\text{visited} \leftarrow \text{False}$ 
4   end
5    $\text{time} \leftarrow 1$ 
6   for  $v \in V$  do
7     if not  $v.\text{visited}$  then
8        $\pi(v) \leftarrow \text{null}$ 
9       Explore(  $G, v$  )
10    end
11  end

1 Explore( $G, v$ ):
2   Previsit( $v$ )
3   for  $(v, u) \in E$  do
4     if not  $u.\text{visited}$  then
5        $\pi(u) \leftarrow v$ 
6       Explore(  $G, u$  )
7     end
8  end
9  Postvisit( $v$ )

1 Previsit( $v$ ):
2    $\text{pre}(v) \leftarrow \text{time}$ 
3    $\text{time} \leftarrow \text{time} + 1$ 

1 Postvisit( $v$ ):
2    $\text{post}(v) \leftarrow \text{time}$ 
3    $\text{time} \leftarrow \text{time} + 1$ 

```

Properties of depth-first search. Depth-first search yields valuable information about the structure of a graph. Perhaps the most basic property of depth-first search is that the predecessor subgraph G_π does indeed form a forest of trees since the structure of the depth-first trees exactly mirrors the structure of recursive calls of explore-function. That is, $u = \pi(v)$ if and only if $\text{explore}(G, v)$ was called during a search of u 's adjacency list.

Additionally, vertex v is a descendant of vertex u in the depth-first forest if and only if v is discovered during the time in which $\text{pre}(u)$ has already been set but $\text{post}(u)$ hasn't¹. Another important property of depth-first search is that discovery and finish times have a parenthesis structure. If the explore procedure were to print a left parenthesis " u " when it discovers vertex u and to print a right parenthesis " u " when it finishes u , then the printed expression would be well-formed in the sense that the parentheses are properly nested.

The following theorem provides another way to characterize the parenthesis structure.

Theorem 1 (Parenthesis theorem). *In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:*

1. *the intervals $[\text{pre}(u), \text{post}(u)]$ and $[\text{pre}(v), \text{post}(v)]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest.*
2. *the interval $[\text{pre}(u), \text{post}(u)]$ is contained entirely within the interval $[\text{pre}(v), \text{post}(v)]$, and u is a descendant of v in a depth-first tree, or*

¹In terms of the presentation from last week, when u 's color is gray

3. the interval $[pre(v), post(v)]$ is contained entirely within the interval $[pre(u), post(u)]$, and v is a descendant of u in a depth-first tree.

Let us use the following notation: we say that line (x) of the recursive call 'Explore(G, v)' is executed at time t if at time t the running of the algorithm got to line number x of Explore where the vertex argument is v . Notice that since Explore is called **only once** on each vertex, the notation is well defined.

Proof. It's enough to prove that for any pair of vertices v, u , the inequality $pre(v) < pre(u) < post(v)$ implies $pre(v) < pre(u) < post(u) < post(v)$.

Consider the call 'Explore(G, v)'. Since the algorithm sets the value of $post(v)$ at line number (9), and it makes all of its recursive calls between lines (3)-(8), we have that the algorithm returns from its recursive calls before time $post(v)$. Furthermore, any line that has been executed during times $[pre(v), post(v)]$ is either a line of the call 'Explore(G, v)' or a line of one of the recursive calls.

In particular, we have that the line which sets $pre(u)$ is executed during times $[pre(v), post(v)]$, and therefore as part of a recursive call. In addition, because the recursive call was returned during the time interval, the assignment of $post(u)$ is made before $post(v)$. Thus, we have both $pre(v) < pre(u) < post(u) < post(v)$ and that u is a descendant of v in π ².

□

Corollary 1. Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $pre(u) < pre(v) < post(v) < post(u)$.

2 DFS on DAGS

the pre is well calibrated on DAGS.

Claim 1. Let $G = (V, E)$ be a DAG, and let $v, u \in V$ such $u \rightarrow v$. Then for any $w \in V$ the call **DFS**(G, w) assigns $post(u) > post(v)$.

Claim 2. Define the relation $v \sim u$ to holds if u and v are on the same strong connected components. Then:

1. $\sim$ is an equivalence relation.
2. Define the graph $G' = (V', E')$ such that V' are the equivalence class respecting to $\sim$ and $\{u, v\} \in E'$ if there is a directed path in G from a vertex in u into a vertex in v . Then G' is a DAG.

Claim 3. Let u the vertex with the minimal post value, then **Explore**(G, u) colorize only the vertices on u 's strong connected components.

²The algorithm defines π to be the recursive tree